一、实验内容

（一）pytorch 基础练习

1.定义数据

（1）基础维度张量创建

代码：

```
import torch

# 可以是一个数
x = torch.tensor(666)
print(x)

# 可以是一维数组（向量）
x = torch.tensor([1,2,3,4,5,6])
print(x)

# 可以是二维数组（矩阵）
x = torch.ones(2,3)
print(x)

# 可以是任意维度的数组（张量）
x = torch.ones(2,3,4)
print(x)
```

执行结果：

```
tensor(666)
tensor([1, 2, 3, 4, 5, 6])
tensor([[1., 1., 1.],
        [1., 1., 1.]])
tensor([[[1., 1., 1., 1.],
         [1., 1., 1., 1.],
         [1., 1., 1., 1.]],

        [[1., 1., 1., 1.],
         [1., 1., 1., 1.],
         [1., 1., 1., 1.]]])
```

（2）特殊初始化张量创建

代码：

```
#  创建一个空张量
x  =  torch.empty(5,3)
print(x)

#  创建一个随机初始化的张量
x  =  torch.rand(5,3)
print(x)

#  创建一个全0的张量，里面的数据类型为  Long
x  =  torch.zeros(5,  3,  dtype=torch.long)
print(x)
```

执行结果：

```
tensor([[5.7722e-21, 0.0000e+00, 0.0000e+00],
        [0.0000e+00, 4.4474e-01, 8.4121e-01],
        [6.5773e-01, 5.4869e-01, 2.5732e-01],
        [4.5158e-01, 7.9182e-01, 5.0401e-01],
        [5.0813e-01, 9.7425e-01, 1.7472e-01]])
tensor([[0.8001, 0.8792, 0.2948],
        [0.6675, 0.9091, 0.5755],
        [0.5051, 0.6399, 0.0795],
        [0.1191, 0.7085, 0.5539],
        [0.2623, 0.7026, 0.4494]])
tensor([[0, 0, 0],
        [0, 0, 0],
        [0, 0, 0],
        [0, 0, 0],
        [0, 0, 0]])
```

(3) 基于现有张量创建

代码:

```
#  基于现有的tensor，创建一个新tensor，
#  从而可以利用原有的tensor的dtype，device，size之类的属性信息
y  =  x.new_ones(5,3)      #tensor  new_*  方法，利用原来tensor的dtype，device
print(y)

z  =  torch.randn_like(x,  dtype=torch.float)
#  利用原来的tensor的大小，但是重新定义了dtype
print(z)
```

执行结果:

```
tensor([[1, 1, 1],
        [1, 1, 1],
        [1, 1, 1],
        [1, 1, 1],
        [1, 1, 1]])
tensor([[ 2.6336, -1.3291,  2.2657],
        [ 0.6584, -0.2032, -0.0980],
        [-1.4241, -0.7882,  0.0325],
        [-1.5508,  2.3147,  2.1483],
        [ 0.7359,  0.3915, -0.6336]])
```

（4）序列张量创建

代码：

```
# Create tensor of numbers from 1 to 5
# 注意这里结果是1到4，没有5
v = torch.arange(1, 5)
print(v)

# returns a 1D tensor of steps equally spaced points between start=3, end=8 and steps=20
torch.linspace(3, 8, 20)
```

执行结果：

```
tensor([1, 2, 3, 4])
tensor([3.0000, 3.2632, 3.5263, 3.7895, 4.0526, 4.3158, 4.5789, 4.8421, 5.1053,
        5.3684, 5.6316, 5.8947, 6.1579, 6.4211, 6.6842, 6.9474, 7.2105, 7.4737,
        7.7368, 8.0000])
```

2.定义操作

（1）张量属性查看与索引切片

代码：

```
# 创建一个 2x4 的tensor
m = torch.Tensor([[2, 5, 3, 7],
                  [4, 2, 1, 9]])

print(m.size(0), m.size(1), m.size(), sep=' -- ')

# 返回 m 中元素的数量
print(m.numel())

# 返回 第0行，第2列的数
print(m[0][2])

# 返回 第1列的全部元素
print(m[:, 1])

# 返回 第0行的全部元素
print(m[0, :])
```

执行结果：

```
2 -- 4 -- torch.Size([2, 4])
8
tensor(3.)
tensor([5., 2.])
tensor([2., 5., 3., 7.])
```

(2) 张量基础运算

代码:

```
# Scalar product
m @ v.float()

# Calculated by 1*2 + 2*5 + 3*3 + 4*7
m[[0], :] @ v.float()

# Add a random tensor of size 2x4 to m
m + torch.rand(2, 4)
```

执行结果:

```
tensor([[2.3858, 5.8129, 3.6609, 7.1043],
        [4.9043, 2.8488, 1.6344, 9.6475]])
```

(3) 张量转置与拼接

代码:

```
# 转置，由 2x4 变为 4x2
print(m.t())
# 使用 transpose 也可以达到相同的效果，具体使用方法可以百度
print(m.transpose(0, 1))

# 创建两个 1x4 的tensor
a = torch.Tensor([[1, 2, 3, 4]])
b = torch.Tensor([[5, 6, 7, 8]])

# 在 0 方向拼接 （即在 Y 方各上拼接），会得到 2x4 的矩阵
print( torch.cat((a,b), 0))

# 在 1 方向拼接 （即在 X 方各上拼接），会得到 1x8 的矩阵
print( torch.cat((a,b), 1))
```

执行结果：

```
tensor([[2., 4.],
        [5., 2.],
        [3., 1.],
        [7., 9.]])
tensor([[2., 4.],
        [5., 2.],
        [3., 1.],
        [7., 9.]])
tensor([[1., 2., 3., 4.],
        [5., 6., 7., 8.]])
tensor([[1., 2., 3., 4., 5., 6., 7., 8.]])
```

（4）张量可视化与 NumPy 转换

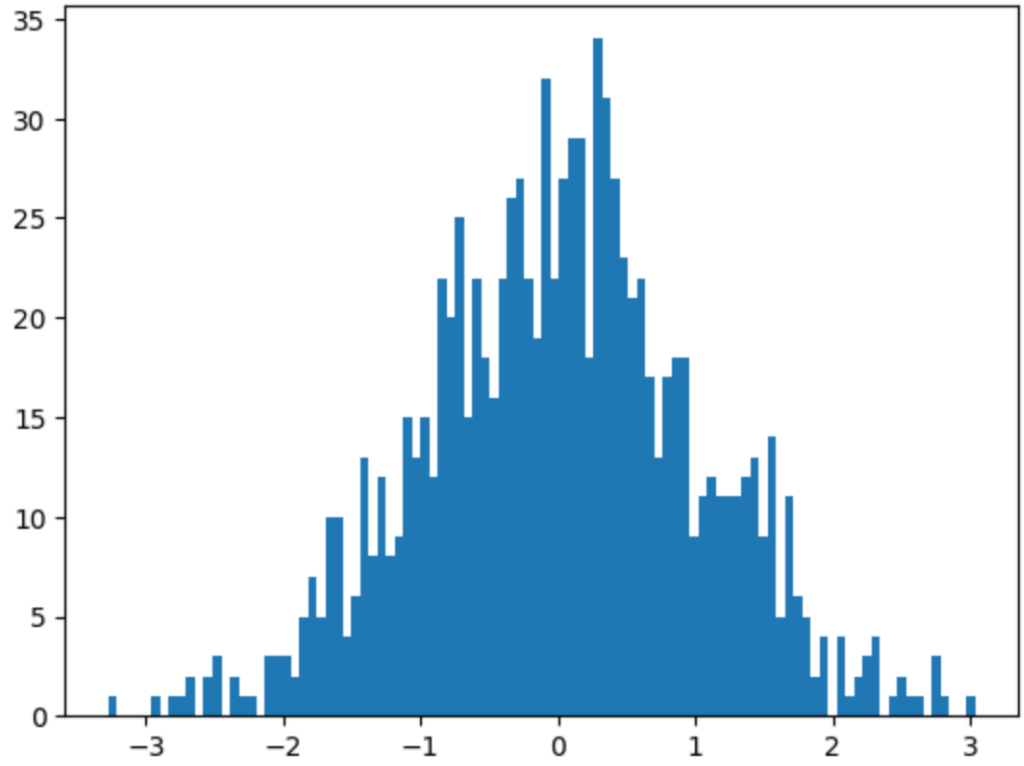
代码：

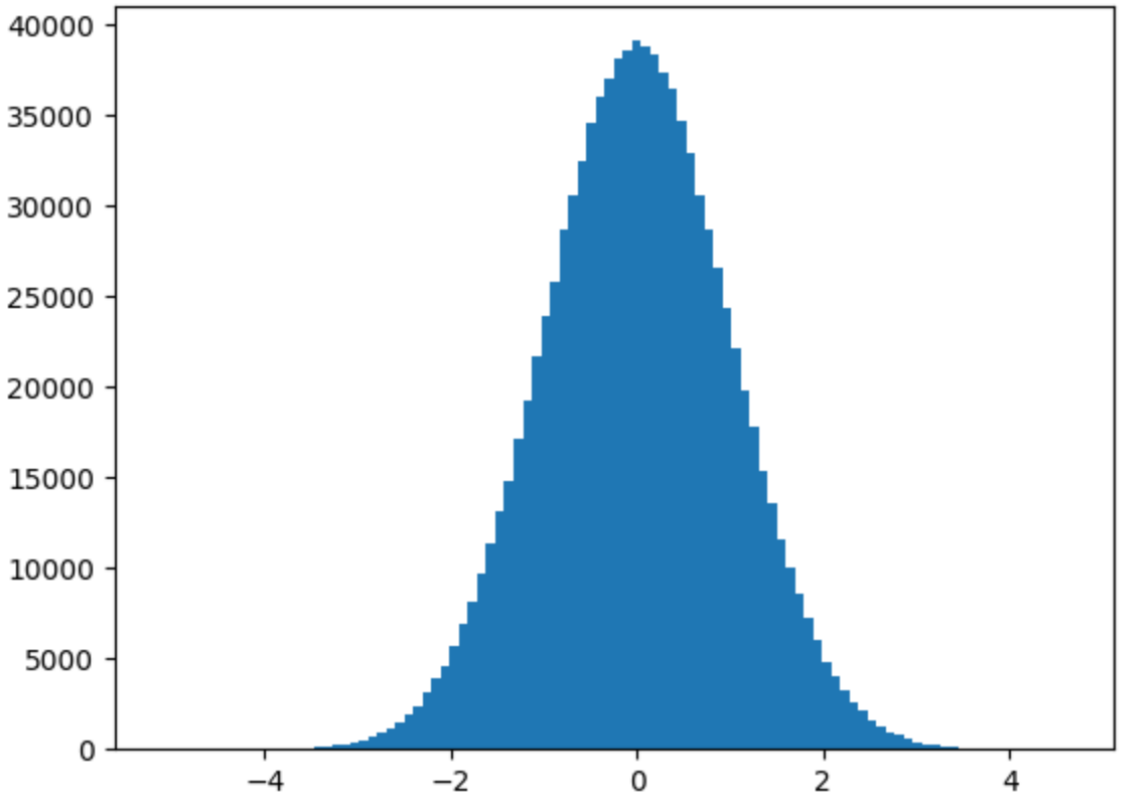
```
from matplotlib import pyplot as plt

# matlabplotlib 只能显示numpy类型的数据，下面展示了转换数据类型，然后显示
# 注意 randn 是生成均值为 0， 方差为 1 的随机数
# 下面是生成 1000 个随机数，并按照 100 个 bin 统计直方图
plt.hist(torch.randn(1000).numpy(), 100);
```

```
# 当数据非常非常多的时候，正态分布会体现的非常明显
plt.hist(torch.randn(10**6).numpy(), 100);
```

执行结果：





思考总结：

这次 PyTorch 基础练习动手跑下来，把张量创建、索引切片、矩阵运算和维度拼接这些基础操作都过了一遍。过程里也遇到一些问题，主要是版本写法和数据类型不匹配，后续都一一改正了。真切感受到这些看似简单的张量操作就是深度学习的底层基石，把维度规则、数据类型这些细节摸扎实，后面搭建网络才能少在基础问题上卡壳。

（二）螺旋数据分类

1.环境准备与螺旋数据生成

代码：

```
#  下载绘图函数到本地。（画点的过程中要用到里面的一些函数）
!wget  https://raw.githubusercontent.com/Atcold/NYU-DLSP21/refs/heads/master/res/plot_lib.py

#  引入基本的库，然后初始化重要参数
import  random
import  torch
from  torch  import  nn,  optim
import  math
from  IPython  import  display
from  plot_lib  import  plot_data,  plot_model,  set_default

#  因为colab是支持GPU的，torch  将在  GPU  上运行
device  =  torch.device("cuda:0"  if  torch.cuda.is_available()  else  "cpu")
print('device:  ',  device)

#  初始化随机数种子。神经网络的参数都是随机初始化的，
#  不同的初始化参数往往会导致不同的结果，当得到比较好的结果时我们通常希望这个结果是可以复现的，
#  因此，在pytorch中，通过设置随机数种子也可以达到这个目的
seed  =  12345
random.seed(seed)
torch.manual_seed(seed)

N  =  1000    #  每类样本的数量
D  =  2          #  每个样本的特征维度
C  =  3          #  样本的类别
H  =  100      #  神经网络里隐层单元的数量
```

```
#  初始化X和Y。X可以理解为特征矩阵，Y可以理解为样本标签
X  =  torch.zeros(N  *  C,  D).to(device)
Y  =  torch.zeros(N  *  C,  dtype=torch.long).to(device)
for  c  in  range(C):
        index  =  0
        t  =  torch.linspace(0,  1,  N)  #  在[0,  1]间均匀的取10000个数，赋给t
        #  下面的代码不用理解太多，总之是根据公式计算出三类样本（可以构成螺旋形）
        #  torch.randn(N)  是得到  N  个均值为0，方差为  1  的一组随机数，注意要和  rand  区分开
        inner_var  =  torch.linspace(  (2*math.pi/C)*c,  (2*math.pi/C)*(2+c),  N)  +  torch.randn(N)  *  0.2

        #  每个样本的(x,y)坐标都保存在  X  里
        #  Y  里存储的是样本的类别，分别为  [0,  1,  2]
        for  ix  in  range(N  *  c,  N  *  (c  +  1)):
                X[ix]  =  t[index]  *  torch.FloatTensor((math.sin(inner_var[index]),  math.cos(inner_var[index])))
                Y[ix]  =  c
                index  +=  1

print("Shapes:")
print("X:",  X.size())
print("Y:",  Y.size())

#  visualise  the  data
plot_data(X,  Y)
```

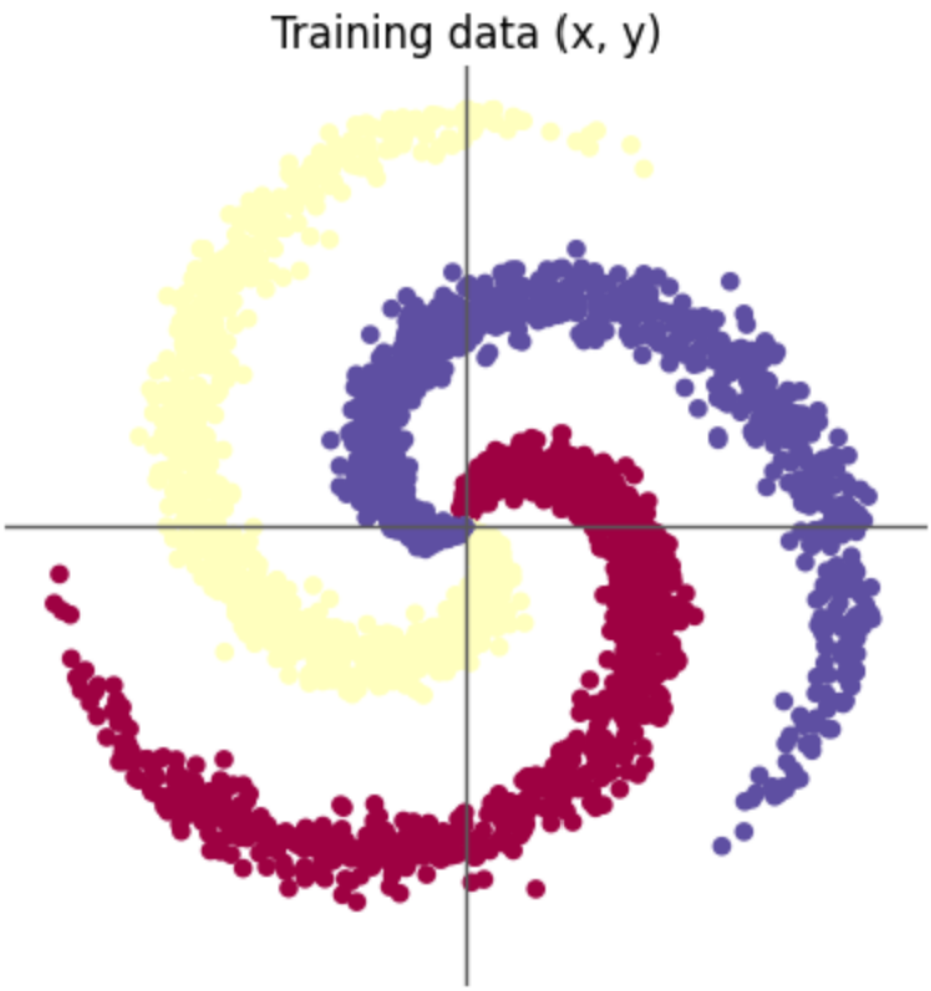
执行结果：

```
--2026-10-10 07:58:29--  https://raw.githubusercontent.com/Atcold/NYU-DLSP21/refs/heads/master/res/plot_lib.py
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 4605 (4.5K) [text/plain]
Saving to: ‘plot_lib.py’

plot_lib.py          100%[===================>]   4.50K  --.-KB/s    in 0s

2026-10-10 07:58:29 (38.1 MB/s) - ‘plot_lib.py’ saved [4605/4605]

device:  cpu
Shapes:
X: torch.Size([3000, 2])
Y: torch.Size([3000])
<matplotlib.collections.PathCollection at 0x791ece5d3380>
```



2.构建线性模型分类

代码：

```
learning_rate = 1e-3
lambda_l2 = 1e-5
# nn 包用来创建线性模型
# 每一个线性模型都包含 weight 和 bias
model = nn.Sequential(
        nn.Linear(D, H),
        nn.Linear(H, C)
)
# 把模型放到GPU上
model.to(device)
# nn 包含多种不同的损失函数，这里使用的是交叉熵（cross entropy Loss）损失函数
criterion = torch.nn.CrossEntropyLoss()

# 这里使用 optim 包进行随机梯度下降(stochastic gradient descent)优化
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate, weight_decay=lambda_l2)

# 开始训练
for t in range(1000):
        # 把数据输入模型，得到预测结果
        y_pred = model(X)
        # 计算损失和准确率
        loss = criterion(y_pred, Y)
        score, predicted = torch.max(y_pred, 1)
        acc = (Y == predicted).sum().float() / len(Y)
        print('[EPOCH]: %i, [LOSS]: %.6f, [ACCURACY]: %.3f' % (t, loss.item(), acc))
        display.clear_output(wait=True)

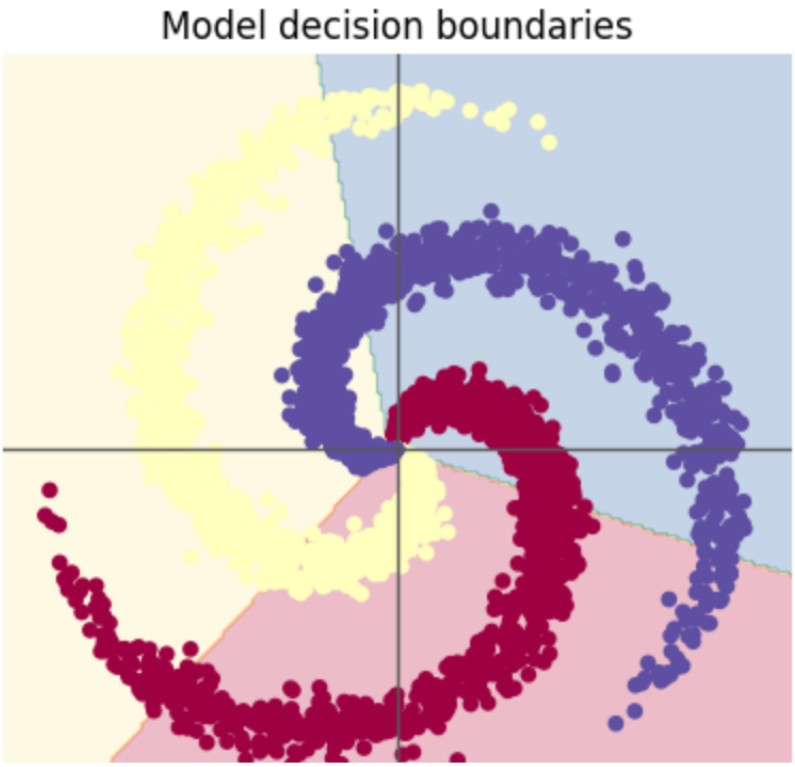

        # 反向传播前把梯度置 0
        optimizer.zero_grad()
        # 反向传播优化
        loss.backward()
        # 更新全部参数
        optimizer.step()

# 查看预测结果维度与单样本详情
print(y_pred.shape)
print(y_pred[10, :])
print(score[10])
print(predicted[10])

# Plot trained model
print(model)
plot_model(X, Y, model)
```

执行结果：

```
torch.Size([3000, 3])
tensor([-0.2245, -0.2594, -0.2080], grad_fn=<SelectBackward0>)
tensor(-0.2080, grad_fn=<SelectBackward0>)
tensor(2)
Sequential(
  (0): Linear(in_features=2, out_features=100, bias=True)
  (1): Linear(in_features=100, out_features=3, bias=True)
)
```



3.构建两层神经网络分类

代码：

```
learning_rate = 1e-3
lambda_l2 = 1e-5

# 这里可以看到，和上面模型不同的是，在两层之间加入了一个 ReLU 激活函数
model = nn.Sequential(
        nn.Linear(D, H),
        nn.ReLU(),
        nn.Linear(H, C)
)
model.to(device)

# 下面的代码和之前是完全一样的，这里不过多叙述
criterion = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate, weight_decay=lambda_l2)

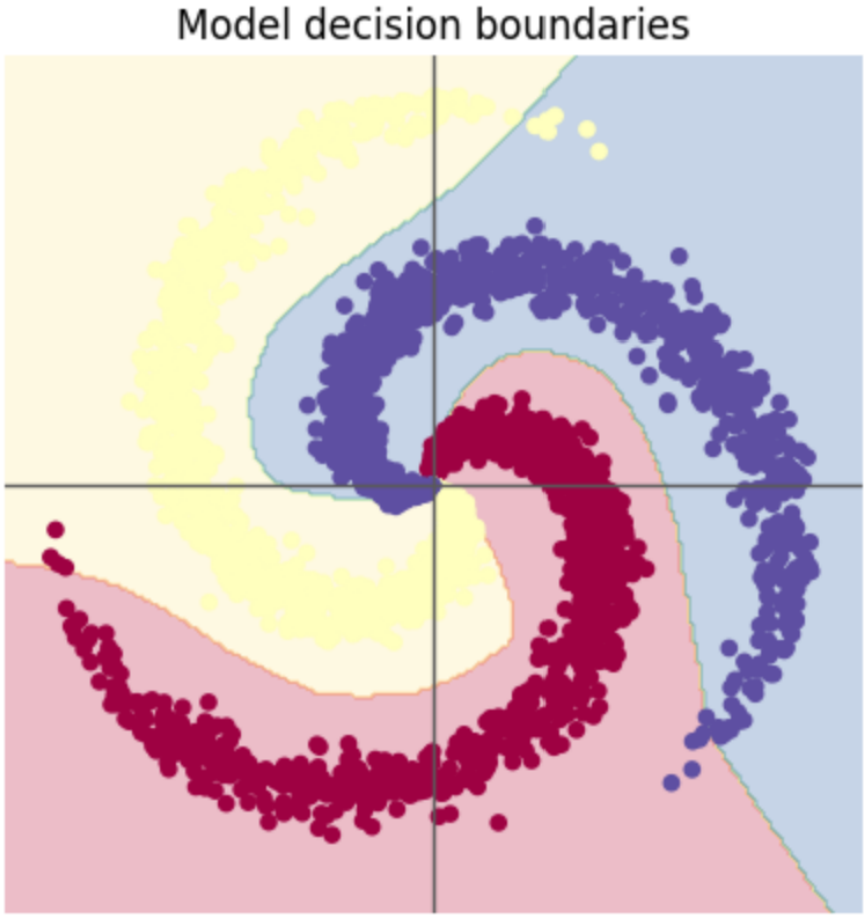
for t in range(1000):
        y_pred = model(X)
        loss = criterion(y_pred, Y)
        score, predicted = torch.max(y_pred, 1)
        acc = ((Y == predicted).sum().float() / len(Y))
        print("[EPOCH]: %i, [LOSS]: %.6f, [ACCURACY]: %.3f" % (t, loss.item(), acc))
        display.clear_output(wait=True)
```

```
        # zero the gradients before running the backward pass.
        optimizer.zero_grad()
        # Backward pass to compute the gradient
        loss.backward()
        # Update params
        optimizer.step()

# Plot trained model
print(model)
plot_model(X, Y, model)
```

执行结果：

```
Sequential(
  (0): Linear(in_features=2, out_features=100, bias=True)
  (1): ReLU()
  (2): Linear(in_features=100, out_features=3, bias=True)
)
```



思考总结：

这次螺旋数据分类实验最直观的感受就是激活函数的作用，纯线性模型准确率只能卡在 50% 左右，分类边界只能是直线，完全没法应对螺旋这种非线性分布的数据，而加入 ReLU 激活函数的两层神经网络准确率直接提升到 90% 以上，能拟合出弯曲的分类边界，很真切地体会到非线性变换对深度学习的意义。同时也完整走了一遍模型训练的标准流程，前向计算损失、反向传播梯度、迭代更新参数，也注意到了每次迭代前清零梯度的细节，以及 Adam 优化器相比 SGD 在收敛速度和最终效果上的优势，把课上学的理论点通过可视化的方式落地了。

二、问题总结与体会

1、AlexNet有哪些特点？为什么可以比LeNet取得更好的性能？

AlexNet 是 2012 年提出的经典卷积网络，和早期的 LeNet 相比，核心特点有：网络深度更深、首次使用 ReLU 激活函数、加入 Dropout 和数据增强抑制过拟合、支持 GPU 并行训练，另外还有重叠池化、局部响应归一化等细节设计。

它性能超过 LeNet 的核心原因是网络容量和特征提取能力的全面升级。LeNet 是为手写数字识别这种简单小任务设计的，层数少、参数少，只能学习边缘、简单形状这类基础特征。而 AlexNet 更深的结构配合 ReLU 解决了深层训练难的问题，能从大规模图片里分层提取从纹理到物体的复杂特征；Dropout 和数据增强又保证了模型在大数据下不会过拟合，最终在复杂的图像分类任务上准确率远超 LeNet。

2、激活函数有哪些作用？

最核心的作用是给神经网络引入非线性。如果没有激活函数，每一层都只是输入的线性计算，不管叠多少层网络，最终效果都和一层线性模型一样，只能拟合简单的直线规律，根本学不了复杂的数据模式。有了激活函数，网络才能表达各种非线性关系，具备拟合复杂函数的能力。

除此之外还有辅助作用：比如把输出值压缩到固定范围（像 sigmoid 把输出限制在 0~1），方便后续计算；像 ReLU 这类激活函数能让部分神经元不工作，带来稀疏性，既减少计算量，也能一定程度缓解过拟合。

3、梯度消失现象是什么？

梯度消失是训练深层网络时的常见问题，发生在反向传播过程中。

训练网络时，损失的梯度会从输出层一层一层往输入层传，每层更新参数都依赖这个梯度。但如果激活函数的导数小于 1（比如 sigmoid 函数，导数最大只有 0.25），每经过一层，梯度就要乘一个小于 1 的数，层数一多，梯度会越乘越小，传到最前面的浅层时，梯度几乎就变成 0 了。

最终的结果就是网络前面的层参数几乎不更新、学不到东西，只有后面几层在训练，整个网络效果会很差。简单说就是梯度 “传着传着就没了”，浅层网络训练不动。

4、神经网络是更宽好还是更深好？

适当加深比单纯加宽效率更高。

“更宽” 是指单层神经元数量多，好处是单层能容纳更多特征，但参数会暴涨，很容易过拟合，而且特征提取没有层次感。

“更深” 是指网络层数多，优势是特征抽象效率更高：浅层提取边缘、纹理这类简单特征，中层组合成部件，高层组合成完整的物体概念，分层学习的能力很强，能用更少的参数拟合更复杂的规律。

但也不是越深越好，层数太多更容易出现梯度消失、梯度爆炸，训练难度会陡增，甚至出现网络退化。实际应用里要根据任务难度、算力平衡，优先保证合理的深度，再配合合适的宽度。

5、为什么要使用Softmax？

Softmax 一般用在分类任务的输出层，主要有两个原因：

第一，它能把网络输出的任意实数，转换成0 到 1 之间的概率值，并且所有类别的概率加起来刚好等于 1。这样输出就有了明确的概率意义，我们能直观看到每个类别的置信度，比如 “这张图是猫的概率 90%”。

第二，它通过指数运算放大了输出差异：原本数值大的输出，经过指数后会变得更大，对应概率更高；数值小的会变得更小。这样模型的分类结果更明确，也能和交叉熵损失函数很好地配合，方便计算分类误差。

6、SGD 和 Adam 哪个更有效?

Adam 更好、效果更稳定。

SGD 是随机梯度下降，每次用单个样本更新参数，逻辑简单，调参到位的话最终泛化性能可能不错，但它对学习率特别敏感，收敛速度慢，很容易卡在局部最优。

Adam 是自适应学习率优化器，相当于在 SGD 基础上加了动量和自动调整学习率的机制，收敛速度快很多，对学习率的选择没那么苛刻，训练过程更平稳，不容易翻车。